~ Mid-Term Master Notes ~

C++ Programming

Units I, II & III · Simple · Clear · Exam-Ready ✍️

Concepts | Syntax | Programs | Exam Practice

★ 3-Pass Study Method

1. **Understand** the idea + diagram first. Syntax later.
2. **Type & run** the example. **Predict the output** before running.
3. **Close the notes** – write the definition, syntax and one example from memory.

⚙ Compile & Run (GCC / Linux / WSL)

```
g++ -std=c++17 -Wall -Wextra -pedantic main.cpp -o main
./main              // Windows: main.exe
```

Warnings are not always errors, but they often reveal real mistakes!

Code-reading rule

For every program find: **(1) declaration → (2) initialization → (3) operation → (4) output**. For pointers, always **draw the variable and its target**.

UNIT I

Concepts, Basics & Functions

Concept	Meaning in one line	Remember
Class	Blueprint / user-defined type	Declares members + access rules
Object	Instance of a class	Has own non-static data
Encapsulation	Data + functions bundled together	Access control protects state
Overloading	Same name, different parameters	Return type alone ✗
Reference	Alias of an existing object	Must be initialized, can't be reseated
Recursion	Function calls itself	Needs base case + progress

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF

1. Paradigms & OOP

Programming Paradigm

A general **style** of designing a program. **Procedural** = built around functions. **OOP** = built around **objects** (data + behaviour). C++ is **multi-paradigm** (procedural, OOP, generic...).

Main OOP ideas

Idea	Simple explanation	Example
Class & Object	Class = design, object = real usable thing	Class `Student`; objects `s1, s2`
Encapsulation	Keep data + functions together; control access	private data, public methods
Abstraction	Show only what's needed, hide details	`account.deposit(500)`
Inheritance	New class made from existing one (reuse)	ElectricCar from Vehicle
Polymorphism	One name, many behaviours	Overloading = compile-time; virtual = run-time
Message passing	Objects talk by calling public methods	`cart.addItem(item)`

Scope note

Inheritance & polymorphism are not detailed units in this syllabus – know the **definitions**, focus on listed topics first.

Procedural vs OOP

Procedural	Object-Oriented
Organized around functions	Organized around classes & objects
Data passed around separately	Data + operations grouped in a class
Data protection depends on design	Access specifiers give controlled access
Good for small, algorithm tasks	Good for modelling complex systems
`calculateAverage(marks)`	Student object → `calculateAverage()`

✔ Advantages of OOP

✘ Limitations

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF

Modular design · code reuse · controlled access · models real world · easy maintenance

Extra design effort · too complex for tiny tasks · bad class design → tightly coupled code

2. C++ Input / Output

```
#include <iostream>
using namespace std;
int main() {
    int age{};                          // zero-initialized
    cout << "Enter your age: ";
    cin >> age;
    cout << "Age = " << age << '\n';
    return 0;
}
```

➜ `<<` = **insertion** operator (output) | `>>` = **extraction** operator (input)

➜ `main` = entry point; `#include <iostream>` gives I/O declarations.

Expression	Meaning
`cin >> x;`	Read one value (skips leading whitespace)
`cin >> a >> b;`	Read two values, whitespace-separated
`cout << "x=" << x;`	Chain outputs left → right
`cout << '\n';`	New line (does NOT necessarily flush)
`getline(cin, name);`	Read a full line incl. spaces
`cin.ignore(...);`	Discard characters – use before getline after cin >>

```
cin >> roll;
cin.ignore(numeric_limits<streamsize>::max(), '\n');  // clear leftover \n
getline(cin, name);
```

⚠ Common Input Bug

After `cin >> number` the **newline stays** in the buffer. A following `getline` reads that empty line. Fix: `cin.ignore(...)`.

Streams

Stream = flow of characters/bytes between program and a source/destination.

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF

Stream	Purpose	Use
cin	Standard input	Keyboard
cout	Standard output	Normal messages
cerr	Standard error (usually unbuffered)	Errors
clog	Standard error (usually buffered)	Logs
ifstream	Input file stream	Read file
ofstream	Output file stream	Write file
fstream	Input + output file stream	Read & write

`<< and >> are overloaded` – same symbol works for int, double, char, string.

3. Classes, Objects & Access

```
class Student {
private:
    int roll{};  double marks{};
public:
    void setData(int r, double m);
    void display() const;
};
void Student::setData(int r, double m){ roll = r; marks = m; }
void Student::display() const{ cout << roll << " " << marks << '\n'; }

int main(){ Student s1; s1.setData(12, 89.5); s1.display(); }
```

★ **Data members** = object's state (roll, marks). **Member functions** = operations.

★ `::` scope-resolution → define a function **outside** the class.

★ `.` dot operator → access member through an object.

★ `const` after parameter list = function promises **not to modify** the object.

Specifier	Accessible from	Default in
public	Anywhere the object is visible	**struct**
private	Only class members & friends	**class**
protected	Members, friends & derived classes	neither

Encapsulation example

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF

Keep `balance` private in `BankAccount`. Give `deposit()` / `withdraw()` that validate amounts → no one can set a negative balance directly.

★ **Declaration** = introduces name & type. **Definition** = gives the full body/entity.

★ Each object has its **own copy** of non-static data members.

★ Private member can't be used from `main` – use public interface or friendship.

```
class Rectangle {
    double length{}, width{};
public:
    void setDimensions(double l, double w){ if(l>0 && w>0){ length=l; width=w;
    double area() const{ return length*width; }
};
// r.setDimensions(5,3); r.area() → 15
```

4. struct, union, enum

struct vs class

Both define class types (can have constructors, functions, static members). **Only difference: default access** → struct = **public**, class = **private**.

Type	Memory idea	Use when
struct / class	Each object has separate fields	Several values together
union	Members **overlap** in same memory	Low-level; only one active at a time
enum / enum class	One of fixed named constants	States, modes, directions

```
union Number { int i; float f; };
Number n; n.i = 10;  n.f = 3.5f;    // now f is active; i is overwritten

enum Day { Monday, Tuesday, Wednesday };        Day today = Tuesday;
enum class Status { Pending, Running, Done };   Status s = Status::Running;
```

⚠ Union caution

Don't read an **inactive** member as a normal field – writing one overwrites the other.

Traditional enum: names go in surrounding scope, converts to int. enum class: names stay under type (`Status::Done`), no implicit int conversion.

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF

5. Inline & Non-inline Member Functions

Inline function

Compiler **may** paste the body at the call site (optimization). It's a **request**, not a guarantee. It does **not** always make code faster.

Inside class (implicitly inline)

```
class Circle {
  double radius{};
public:
  double area() const
  { return 3.14159*radius*radius; }
};
```

Outside class (not inline)

```
class Circle {
  double radius{};
public:
  void setRadius(double r);
};
void Circle::setRadius(double r)
{ radius = r; }
```

Exam wording ✍️

Don't write "inline always makes code faster." Write: **"inline suggests inline treatment; actual optimization is compiler-dependent."** Inside-class = short ops; outside-class = keeps interface clean for longer code.

6. Static Members

Static data member

Belongs to the **class**, not each object. One shared copy for all objects.

```
class Counter {
public:
    static int count;
    Counter(){ ++count; }
};
int Counter::count = 0;      // definition outside class (pre-C++17 style)
// Counter a,b,c;  cout << Counter::count;  → 3
```

C++17: `inline static int count = 0;` can be initialized inside the class.

Static member function

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF

```
class MathTool { public: static int square(int x){ return x*x; } };
int answer = MathTool::square(5);    // 25
```

> **Remember**
> Static function has **no** `this` **pointer** → can use static members directly, **cannot** touch a particular object's non-static members. Call with `ClassName::function()`.

7. Default Arguments & Manipulators

Default argument = value used when the caller omits it. Defaults go at the **right end** of the list.

```
int power(int exponent, int base = 10) {
    int result = 1;
    for(int i=0;i<exponent;++i) result *= base;
    return result;
}
// power(2) → 100       power(2,3) → 9
```

> **Rules**
> f(int a, int b = 4) ✔ → f(3) means f(3,4). f(int a = 1, int b) ✘ (non-default after default). Put the default in the **declaration once**, not again in the definition.

Inline recap: `inline int cube(int x){ return x*x*x; }` – good for tiny, frequently-called functions.

Manipulators (change stream formatting)

Manipulator	Header	Effect
endl	iostream	New line + flush
setw(n)	iomanip	Width for next field
setprecision(n)	iomanip	Precision (with `fixed` = digits after decimal)
fixed / scientific	iostream	Fixed-point / scientific notation
setfill(ch)	iomanip	Padding character
left / right	iostream	Alignment in field
boolalpha	iostream	Print bool as true/false

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF

```
double x = 12.34567;
cout << fixed << setprecision(2) << x;        // 12.35
cout << setw(6) << setfill('0') << 42;         // 000042
cout << boolalpha << true;                     // true
```

endl vs \n

Use `\n` for a plain line break. `endl` also **flushes** → extra overhead inside big loops.

8. Function Overloading & Scope

Function Overloading

Many functions share one name if parameter lists differ so the compiler can choose clearly. Compiler checks **count, types, conversions**. No single best match → **ambiguous** → **compile error**.

```
int    add(int a, int b)           { return a+b; }
double add(double a, double b)     { return a+b; }
int    add(int a,int b,int c)     { return a+b+c; }
```

✘ Not valid

`int f(int);` and `double f(int);` differ only by **return type** → cannot overload.

Scope – where a name is visible

Scope	Visible	Example
Local / block	Inside function/block	`int x` inside if
Global	From declaration, outside functions	variable outside all functions
Class	Inside class / via ClassName::	`Student::display`
Parameter	In that function	`sum(int a,int b)`
Loop	Only that loop	`for(int i=0;..)`

```
int value = 100;                // global
int main(){
    int value = 20;             // local hides global
    cout << value;             // 20
    cout << ::value;           // 100  (:: reaches global)
}
```

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF

Overloading	Default arguments
Several definitions, same name, different params	One function; omitted trailing args are filled in
Compiler picks viable overload	Compiler uses declared defaults
Both can become **ambiguous** if mixed carelessly.	

9. Friend Function & Friend Class

Friend

Not a member, but the class **grants it permission** to use private/protected members.

Friendship is **given**, not automatic, not mutual, not inherited.

```
class Box {
    int width;
public:
    explicit Box(int w) : width(w) {}
    friend void showWidth(const Box& b);
};
void showWidth(const Box& b){ cout << b.width; }   // allowed
// call: showWidth(b);   NOT b.showWidth();

class Account { double balance = 0; friend class Auditor; };
class Auditor { public: double inspect(const Account& a) const{ return a.balanc
```

Friend function	Friend class
Access for one named non-member function	Access for all member functions of that class
Useful for symmetric ops / stream operators	Useful for tightly cooperating classes
Doesn't become a member	Doesn't make classes derived from each other

Use friends sparingly – they weaken encapsulation!

10. References & Parameter Passing

Reference variable

An **alias** for an existing object. Must be initialized when declared; afterwards always refers to the same object.

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF

```cpp
int score = 80;
int& alias = score;
alias = 95;        // score becomes 95
```

① Call by value

```cpp
void change(int x){ x = 99; }
// a=10 → still 10
```

② Call by address

```cpp
void change(int* p){
  if(p!=nullptr) *p = 99; }
// change(&a); → a=99
```

③ Call by reference

```cpp
void change(int& x){ x = 99; }            // a becomes 99
void printName(const string& name){ cout << name; }   // no copy, no change
```

Method	Parameter	Call	Changes caller?	Null?
Value	T	f(x)	No	–
Address	T*	f(&x)	Yes (via *p)	Can be nullptr
Reference	T&	f(x)	Yes (alias)	Normally not null

Exam precision ✍️

In C++, arguments are passed **by value unless the parameter is a reference**. "Call by address" passes a **pointer by value**; the pointed-to object can still be modified.

```cpp
void swapValues(int& a, int& b){ int temp = a; a = b; b = temp; }
// x=4, y=9 → swapValues(x,y) → 9 4
```

11. Recursion

Recursion

A function calls itself (directly/indirectly) to solve a **smaller version** of the same problem. Needs a **base case** (stops) + a **recursive step** (moves toward base).

```cpp
long long factorial(int n){
    if(n < 0)  return -1;       // invalid input signal
    if(n <= 1) return 1;        // BASE CASE
```

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF

```
    return n * factorial(n-1);    // RECURSIVE STEP
}
```

```
factorial(4) = 4 × factorial(3)
             = 4 × 3 × factorial(2)
             = 4 × 3 × 2 × factorial(1)
             = 4 × 3 × 2 × 1   = 24
(each pending call waits for the next to return)
```

✔ Recursion checklist

① Base case present? ② Each call moves toward it? ③ Depth limited (no stack overflow)? ④ Result type big enough?

Recursive member function

```
class Counter { public:
  void countdown(int n) const{
    if(n <= 0){ cout << "Go!\n"; return; }
    cout << n << ' ';  countdown(n-1);
  } };
```

Recursion	Iteration
Function calls + stack frames	Loops (for/while/do-while)
Natural for tree-like problems	Easier for simple repetition
Deep recursion can exhaust stack	No per-step stack cost
Needs a correct base case	Needs a terminating condition

⚡ Unit I – Rapid Revision

★ OOP groups data + behaviour; encapsulation protects state; abstraction exposes useful interface.

★ `cin` uses `>>`, `cout` uses `<<`; `getline` reads spaces; `ignore` clears leftover newline.

★ **class** → private default; **struct** → public default.

★ Non-static members = per object; **static** = shared by class.

★ Overload by **parameter list**, never return type alone. Defaults fill trailing args.

★ Friend can access private data but is **not** a member.

★ Value copies · pointer uses address · reference aliases the original.

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF

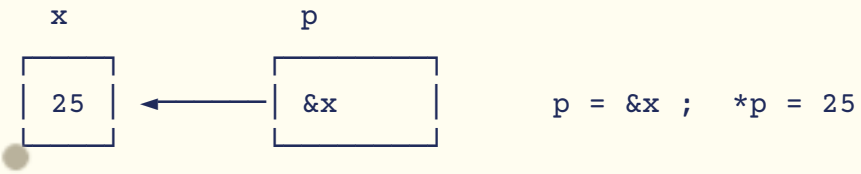
★ Recursion = base case + progress.

UNIT II

Pointers, References, Arrays & Strings

Mental model 🧠

A variable has a **value** and lives in storage. A **pointer** is also a variable – its value is an **address**. Dereferencing a valid pointer reaches the object at that address.



12. Pointer Fundamentals

```
int x = 25;
int* p = &x;
cout << x;       // 25
cout << &x;      // address of x
cout << p;       // same address
cout << *p;      // 25
*p = 40;          // x is now 40
```

Syntax	Meaning
`&x`	Address of x
`int* p`	p is a pointer to int
`p = &x`	Store x's address in p
`*p`	Dereference – the int p points to
`p == nullptr`	Is p null?
`p->member`	Same as `(*p).member`

Declaration pitfall

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF

`int* p, q;` → p is pointer, **q is a plain int**.

`int *p, *q;` → both pointers. (The * belongs to each declarator.)

Null, wild & dangling

Kind	Meaning	Safe handling
Null	Points to nothing: `int* p = nullptr;`	Check before dereferencing
Wild	Uninitialized – random address	Initialize immediately (nullptr / address)
Dangling	Points to object whose life ended / memory freed	Never dereference; reset pointer
Valid	Points to a live object of right type	Respect lifetime & bounds

NEVER DO THIS ☠

Dereferencing uninitialized, dangling or null pointer = **undefined behaviour** (crash, garbage, or "works" then fails later).

```
int* p = nullptr;
if(p != nullptr){ cout << *p; }        // safe
```

Pointer assignment

Copies the **address**, not the object. Both then refer to the same variable.

```
int x = 7;  int* p = &x;  int* q = p;
*q = 15;    cout << x << ' ' << *p;     // 15 15
```

13. void*, Pointer Arithmetic, Pointer-to-Pointer

void pointer

Generic pointer – holds the address of any object type, but **can't be dereferenced directly**; cast first. Arithmetic on `void*` is not standard C++.

```
int n = 42;  void* vp = &n;
int* ip = static_cast<int*>(vp);
cout << *ip;        // 42
```

Pointer arithmetic

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF

`p + 1` moves to the **next element**, not the next byte (step = `sizeof(*p)`).

```
int a[] = {10, 20, 30};
int* p = a;                    // points to a[0]
*p → 10      *(p+1) → 20       p[2] → 30
p++;    *p → 20
```

Expression (p → a[0])	Meaning
`p`	Address of a[0]
`*p`	Value of a[0]
`p + 1`	Address of a[1]
`*(p + 2)` = `p[2]`	Value of a[2]
`a + 3`	One-past-end – **don't dereference**

Trap

Subtracting/comparing pointers only makes sense when both point into the **same array** (or one past it).

Pointer to pointer

```
int x = 5;  int* p = &x;  int** pp = &p;
cout << **pp;      // 5
**pp = 18;          // x = 18
```

```
pp ⟶ p ⟶ x
**pp  = *(*pp) = *p = x
```

Read `**pp` inside-out: `*pp` gives p, `**pp` gives x.

14. Pointers & Class Objects

Pointer to object

```
tudent s;  s.roll = 21;
Student* ps = &s;
ps->show();   cout << ps->roll;       // (*ps).show() is the same
```

Parentheses needed in (*ps).member because . binds tighter than *

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF

The `this` pointer

Inside a **non-static** member function, `this` = pointer to the object that called the function. Static functions have no `this`.

```
void setRoll(int roll){ this->roll = roll; }   // left=member, right=parameter
Student& self(){ return *this; }                // return current object
```

Class containing a pointer

Key question: **who owns the resource?** If the class owns raw dynamic memory, copy & destruction must be designed carefully.

```
class IntBox {
    int* value;
public:
    explicit IntBox(int v) : value(new int(v)) {}
    ~IntBox(){ delete value; }
    int get() const{ return *value; }
};
```

Copying raw pointers

Default copy copies the **address** → two objects own one allocation → **double delete**.

Prefer `std::unique_ptr` or write correct copy operations.

Shallow vs Deep copy

Shallow copy	Deep copy
Copies pointer values (addresses)	Allocates new storage & copies the pointed-to value
Two objects share one allocation	Objects are independent
Can cause double delete / dangling	Needs correct alloc, copy, assign, destroy

```
IntBox(const IntBox& other) : value(new int(*other.value)) {}   // deep copy ct
```

Modern C++ prefers RAII types: `unique_ptr`, `vector`, `string`.

15. Arrays & Multidimensional Arrays

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF

```
int marks[5] = {70, 82, 90, 68, 75};
cout << marks[0];      // 70
marks[2] = 95;
```

★ Same-type elements stored **contiguously**; indices **0 ... n-1**.

★ Built-in arrays have **no bounds checking** (`marks[5]` is out of bounds!).

★ Size is fixed. Passed to a function → becomes a **pointer**; length is not passed.

★ `sizeof(a)/sizeof(a[0])` gives count only in the declaring scope.

```
int a[] = {4,8,2,6};  int sum = 0;
for(int i=0;i<4;++i) sum += a[i];      // 20
```

2D arrays – row-major order

```
int a[2][3] = { {1,2,3}, {4,5,6} };
for(int r=0;r<2;++r){
   for(int c=0;c<3;++c) cout << a[r][c] << ' ';
   cout << '\n';
}
```

```
Memory (row-major):  1 2 3 | 4 5 6
a[r]    → r-th row
a[r][c] → element at row r, column c
a[2][..] or a[..][3] → OUT OF BOUNDS for a[2][3]
```

Passing a 2D array to a function

```
void printMatrix(const int a[][3], int rows){ /* loops */ }
```

Column size (3) must be known so the compiler can calculate element addresses.

Array inside a class

```
class Matrix2x3 {
    int data[2][3]{};
public:
    void set(int r,int c,int v){ if(r>=0&&r<2&&c>=0&&c<3) data[r][c]=v; }
    int  get(int r,int c) const{ if(r>=0&&r<2&&c>=0&&c<3) return data[r][c]; re
};
```

Encapsulates the array with **bounds checks** (returning 0 is just a demo; real code should signal errors).

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF

Array of objects

```
Item items[3];
items[0].id = 101;  items[1].id = 102;  items[2].id = 103;
for(int i=0;i<3;++i) items[i].display();
```

Each element is a full object; constructors run for each when the array is created.

16. std::string

`std::string` (in `<string>`) manages text: assign, join, compare, index, search – with automatic length tracking. Much easier than C-style char arrays.

```
string first = "Ada", last;  last = "Lovelace";
string full = first + " " + last;
cout << full << full.length();
```

Operation	Example	Meaning
Default	`string s;`	Empty string
Literal	`string s = "robot";`	Stores robot
Assign	`s = "C++";`	Replace contents
Copy	`string t = s;`	t gets a copy
Concatenate	`s + t`	Combined string
Append	`s += " lab";`	Add at end

Important member functions

Member	Use	Example
size() / length()	Number of characters	s.size()
empty()	True if no characters	s.empty()
at(pos)	Element access **with bounds check**	s.at(0)
operator[]	Element access, no check	s[0]
append(text)	Add to end	s.append("AI")
push_back(ch)	Add one char	s.push_back('!')
pop_back()	Remove last char	s.pop_back()
insert(pos,text)	Insert text	s.insert(2,"XX")
erase(pos,count)	Remove part	s.erase(1,3)

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF

replace(pos,count,text)	Replace substring	s.replace(0,2,"Hi")
find(text)	Position, or `string::npos` if absent	s.find("robot")
substr(pos,count)	Extract substring	s.substr(2,4)
compare(other)	Lexicographic compare	s.compare(t)

Index starts at **0**. `string::npos` = "not found".

```
string word;  cin >> word;                          // one token (stops at whitesp
cin.ignore(numeric_limits<streamsize>::max(), '\n');
string sentence;  getline(cin, sentence);        // full line with spaces
```

std::string	C-style char array
Size managed dynamically	Fixed capacity
Natural `=` and `+`	Needs functions & care
size()/length()	Scan for null terminator
Preferred for normal C++ text	Needed in low-level/API code

17. Pointer vs Reference & Pointer to Data Member

Pointer	Reference
Stores an **address**	An **alias** of an existing object
May be null	Must be initialized to a valid object
Can be re-pointed	Cannot be reseated
Uses `*` and `->`	Used like the object itself
Pointer arithmetic possible (arrays)	No arithmetic on the reference
Can mean "no object" (nullptr)	Not meant for "absent"

Pointer to data member

Identifies a **non-static data member** of a class **without** naming an object. Use `.*` with an object and `->*` with an object pointer.

```
class Student { public: int roll; };
int Student::* memberPtr = &Student::roll;
Student s{25};  Student* ps = &s;
```

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF

```
s.*memberPtr = 30;          // s.roll = 30
ps->*memberPtr = 31;        // ps->roll = 31
```

Read `int Student::*` as **"pointer to an int member of Student."** It is not an ordinary `int*`.

Distinguish

`int* p = &x;` → points to one particular int. `int Student::* pm = &Student::roll;` → identifies the roll member inside any Student.

⚡ Unit II – Rapid Revision

- ★ `&` gets address · `*` dereferences · `->` member via pointer.
- ★ Always initialize pointers. nullptr: safe to compare, **not** to dereference. Avoid wild & dangling.
- ★ Pointer arithmetic steps by pointed-to type; limited to array range + one-past-end.
- ★ `void*` needs a cast before dereference; no arithmetic.
- ★ `this` = current object inside non-static member functions.
- ★ Arrays start at 0, no bounds check; 2D arrays = arrays of arrays, row-major.
- ★ `std::string`: size, find, substr, append, erase, replace...
- ★ Pointer-to-data-member: `.*` with object, `->*` with pointer.

UNIT III

File Operations, Constructors & Destructors

Core question

How does a program keep data after it ends, and how should an object **start** (constructor) and **clean up** (destructor)?

18. File Handling Basics

Variables vanish when the program ends. A **file** gives **persistent storage**. Use `<fstream>`.

Class	Purpose	Declaration

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF

ifstream	Read from file	`ifstream fin("input.txt");`
ofstream	Write to file	`ofstream fout("output.txt");`
fstream	Read & write	`fstream f("data.txt", ios::in|ios::out);`

```
ofstream fout("notes.txt");
if(!fout){ cerr << "Could not open file\n"; return 1; }
fout << "C++ file handling\n";
fout.close();

ifstream fin("notes.txt");  string line;
while(getline(fin, line)) cout << line << '\n';
```

★ Open via constructor or `open()`. **Always check** that opening succeeded.

★ `close()` ends the file association. Streams also auto-close when destroyed.

File modes

Mode	Meaning	Note
ios::in	Open for reading	File must be readable
ios::out	Open for writing	Usually creates / truncates
ios::app	Append – every write at end	Keeps old content
ios::ate	Start positioned at end	Can seek elsewhere (unlike app)
ios::trunc	Discard existing contents	Use only when replacing
ios::binary	Binary (no text translation)	For byte records
ios::in | ios::out	Read + write	File often must exist
ios::out | ios::app	Append output	Good for logs
ios::out | ios::binary	Binary output	Byte-oriented

Combine modes with bitwise OR `|`. Always check stream state – never assume the file opened.

Useful file functions

Function	Purpose
open(path, mode) / close()	Attach / detach file
is_open()	True if file is open
good()	No error flags set
fail()	Operation failed / bad format
eof()	End-of-file flag (set only after a read passes the end)

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF

bad()	Serious stream error
clear()	Reset error flags (doesn't erase file)
get(ch) / put(ch)	Read / write one character
getline(...)	Read a line
read(buf,n) / write(buf,n)	Read / write n bytes
seekg(pos) / seekp(pos)	Move get (input) / put (output) position
tellg() / tellp()	Report get / put position

End-of-file loop

Prefer `while(getline(fin,line))` or `while(fin >> value)`.

Avoid `while(!fin.eof())` – eofbit is set only after a failed read, so the last value may be processed twice.

19. Reading & Writing Text Files

```
// write
ofstream fout("marks.txt");
fout << 101 << ' ' << "Udit" << ' ' << 92.5 << '\n';
fout.close();

// read
ifstream fin("marks.txt");  int roll{}; string name; double marks{};
if(fin >> roll >> name >> marks) cout << roll << " " << name << " " << marks;
else cerr << "Invalid or missing record\n";
```

Formatted output stores a **text form** (readable). `>>` assumes name has **no spaces**; for spaces, use a delimited format + `getline`.

```
char ch;
while(fin.get(ch)) cout.put(ch);        // char-by-char; get() keeps whitespace
```

Check errors

Failures happen (bad path, permission, disk, wrong format). Check `is_open()`, and for important output check the stream after `close()`: `if(!fout) cerr << "Write/close failed";`

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF

20. Sequential & Random Access

Sequential

Read/write records **in order**, from current position onward. Simple; good for text lines, logs.

```
ifstream fin("numbers.txt"); int
while(fin >> v) cout << v;
```

Random

Jump to **any position**. Streams have a **get pointer** (input) and **put pointer** (output).

```
file.seekg(0, ios::beg);
file.seekp(0, ios::end);
```

Constant	Meaning
ios::beg	Beginning of file
ios::cur	Current position
ios::end	End of file

Fixed-size records

Offset of record `i` (zero-based) = **i × sizeof(Record)** — only valid if the file truly stores fixed-size records.

```
streamoff offset = static_cast<streamoff>(index) * static_cast<streamoff>(sizec
file.seekg(offset, ios::beg);
```

Caution

`sizeof(Record)` isn't a portable file format – **padding, type sizes, endianness** and compiler can change the layout.

21. Binary Files

Text mode	Binary mode
Formatted textual representation	Raw bytes, no newline translation
Human-readable	Not human-readable
Byte length varies for numbers	Byte count controlled by read/write

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF

Good for editable / shareable data	Good for raw buffers & designed formats

```
int number = 1234;
{ ofstream out("number.bin", ios::binary);
  out.write(reinterpret_cast<const char*>(&number), sizeof(number)); }
int recovered = 0;
{ ifstream in("number.bin", ios::binary);
  in.read(reinterpret_cast<char*>(&recovered), sizeof(recovered)); }
cout << recovered;      // 1234
```

Is it safe to write an object as raw bytes?

Only for simple types. A class with **pointers, std::string, virtual functions** or special invariants must not be dumped with `sizeof(obj)` – a pointer's bytes are an address, not the data it points to.

Exam answer ✍️

Binary I/O uses `ios::binary` with `read()` and `write()`. Raw record: `write(reinterpret_cast<const char*>(&obj), sizeof obj)`. Portability and type-safety depend on the object and file format.

22. Files with Structures & Classes

Struct → text file (field by field)

```
struct StudentRecord { int roll; string name; double marks; };
StudentRecord s{101, "Mira", 91.5};
ofstream out("students.txt");
out << s.roll << '\n' << s.name << '\n' << s.marks << '\n';
```

Simple struct → binary (no pointers / strings)

```
struct SimpleRecord { int roll; double marks; };
SimpleRecord s{101, 91.5};
ofstream out("record.bin", ios::binary);
out.write(reinterpret_cast<const char*>(&s), sizeof(s));
```

Still depends on platform padding & byte order – not a universal format.

Class saves itself

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF

```
bool saveText(const string& path) const{
    ofstream out(path);
    if(!out) return false;
    out << roll << '\n' << name << '\n' << marks << '\n';
    return static_cast<bool>(out);
}
```

Member function can use private fields and defines the format. Text is more readable than dumping object bytes.

23. Constructors

Constructor

*Special member function that **initializes an object** at creation. Same name as class, **no return type**, can be **overloaded**.*

Default

```
class Point { int x,y;
public: Point():x(0),y(0){} };
Point p;
```

Callable with no arguments. Once you declare any constructor, don't assume the compiler gives you a no-arg one.

Parameterized

```
Point(int a,int b):x(a),y(b){}
Point p(3,7);
Point q{4,9};
```

With default arguments

```
Point(int a = 0, int b = 0) : x(a), y(b) {}
Point p;        // (0,0)
Point q(5);     // (5,0)
Point r(5,8);   // (5,8)
```

Ambiguity

Don't have both `Point()` and `Point(int x = 0)` – `Point p;` can call either → conflict.

Copy constructor

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF

Creates a **new** object from an existing object of the same class. Signature: `ClassName(const ClassName& other)`.

```
Point(const Point& other) : x(other.x), y(other.y) {}
Point p(2,3);
Point q = p;       // copy construction
```

Copy construction	Copy assignment
Creates a **new** object	Replaces state of an **existing** object
`Point q = p;` (q being created)	`q = p;` (q already exists)
Controlled by copy constructor	Controlled by copy-assignment operator

Shallow-copy hazard & deep copy

```
class Buffer {
    int* data;
public:
    explicit Buffer(int value) : data(new int(value)) {}
    Buffer(const Buffer& other) : data(new int(*other.data)) {}   // DEEP copy
    ~Buffer(){ delete data; }
};
```

Rule of Three

If a class needs a custom **destructor**, **copy constructor** or **copy-assignment**, it usually needs all three. **Rule of Zero**: let standard RAII types manage ownership instead.

24. Member Initializer Lists

Initializes base classes & data members **before** the constructor body. Written after a colon.

```
Rectangle(int l, int w) : length(l), width(w) {}
```

Required for...

✔ **reference** members ✔ **const** members ✔ members with **no default constructor**.
Also better for efficiency & ownership.

Initialization order

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF

Members are initialized in **declaration order** in the class – not the order written in the list. Write the list in declaration order.

```
class Example { int first; int second;
public: Example() : first(1), second(first + 1) {} };    // ok: first declared f

class Record { const int id; int& externalValue;
public: Record(int i, int& v) : id(i), externalValue(v) {} };
```

25. Destructors & Object Lifetime

Destructor

Called when an object's lifetime **ends** (leaves scope / `delete`). Name `~ClassName`, **no return type, no parameters** → only **one** per class (no overloading).

```
class Demo { public:
    Demo(){ cout << "Constructed\n"; }
    ~Demo(){ cout << "Destroyed\n"; } };
int main(){ Demo d; cout << "Inside main\n"; }    // d destroyed at closing }
```

```
Output:  Constructed
         Inside main
         Destroyed
```

Order

- ➜ Member subobjects are constructed in **declaration order**, destroyed in **reverse**.
- ➜ Arrays of objects: destroyed in **reverse** order of construction.

Why destructors matter

- ★ Release resources the object owns (memory, file handle).
- ★ Pair **acquire in constructor** ↔ **release at end of life**.
- ★ Prefer resource-managing types (`string`, `vector`, file streams).

ifstream / ofstream / fstream close their file automatically in the destructor! Explicit close() is still handy to close at a chosen time or check status.

Constructor	Destructor

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF

Initializes a new object	Cleans up when lifetime ends
Same name as class, no return type	~ClassName, no return type
Can be overloaded	Cannot be overloaded; no parameters
May take params / defaults	Takes no parameters
Sets up invariants, acquires resources	Releases owned resources

26. Object + File Example

```
bool save(const string& filename) const{
    ofstream out(filename);
    if(!out) return false;
    out << roll << '\n' << name << '\n' << marks << '\n';
    return static_cast<bool>(out);
}
bool load(const string& filename){
    ifstream in(filename);
    if(!in) return false;
    int r; string n; double m;                          // temporaries first
    if(!(in >> r)) return false;
    in.ignore(numeric_limits<streamsize>::max(), '\n');
    if(!getline(in, n) || !(in >> m)) return false;
    roll = r; name = n; marks = m;                     // update only if ALL read
    return true;
}
```

Design lesson

`load` reads into **temporary variables first**, then updates the object – so a bad file never leaves it half-updated. Text formats are readable & easy to inspect; binary is compact but needs a deliberate, validated format.

⚡ Unit III – Rapid Revision

★ `<fstream>`: ifstream (in), ofstream (out), fstream (both). **Always check open success.**

★ in = read · out = write · app = write at end · ate = start at end · binary = binary mode · trunc = discard contents.

★ Use `while(fin >> v)` / `while(getline(...))`, not `!eof()`.

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF

- ★ `seekg/tellg` = input position · `seekp/tellp` = output position.
- ★ `read()/write()` move bytes; raw object dumps aren't automatically safe/portable.
- ★ Constructor initializes; destructor cleans up.
- ★ Copy construction = new object from another; copy assignment = update existing.
- ★ Initializer lists run before the body, in **declaration** order.
- ★ Raw owning pointers need deliberate copy & destruction; prefer RAII.

EXAM TOOLKIT

Practice · Traps · Cheat Sheet

These are **practice prompts** from your syllabus, not a prediction of the real paper.

Short-answer practice

Unit I

- Define OOP; explain encapsulation, abstraction, polymorphism with examples.
- Procedural vs OOP.
- Class & object; public/private/protected.
- struct vs union vs enum.
- Inline vs non-inline member functions – is inline guaranteed faster?
- Static data members & functions with a program.
- Default arguments & manipulators (setw, setprecision).
- Overloading & scope resolution – why not by return type?
- Friend function vs friend class.
- Call by value / address / reference.
- Recursion – write & trace factorial/sum.

Unit II

- Pointer; &, *, nullptr, assignment.
- Wild vs null vs dangling.
- void*, pointer arithmetic, pointer-to-pointer.
- `this` pointer; `.` vs `->`.
- Shallow vs deep copy.
- 1D & 2D arrays; row-major layout.
- Array of objects; arrays inside a class.
- std::string + six member functions.
- Pointer vs reference.
- Pointer-to-data-member; `.*` and `->*`.

Unit III

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF

- ifstream, ofstream, fstream.
- File modes: in, out, app, ate, trunc, binary.
- open(), close(), is_open(), eof(), fail(), read(), write().
- Sequential vs random access.
- Program to write & read values from a text file.
- Binary file I/O – why dumping any class as raw bytes is unsafe.
- File handling with structure / class.
- Default, parameterized, copy & default-argument constructors.
- Constructor vs destructor; order of execution.
- Member initializer lists & initialization order.

Programming problems

1. Student class: private roll & marks, public setters, display, average/grade function.
2. Function overloading: int, double, and a 3-argument overload.
3. Swap with call by value, pointer & reference – which can change the caller?
4. Recursive factorial; trace for n = 4.
5. Traverse an int array with pointers and compute the sum.
6. Class with static counter incremented per constructed object.
7. std::string: join first + last name, search a substring, extract a substring.
8. 3×3 matrix: print row sums & column sums.
9. Save a student record to a text file and read it back.
10. Class owning a dynamic int with correct deep-copy constructor + destructor.

Output tracing – try before checking!

```
// Q1
int x=4; int* p=&x;
*p = *p + 3;  cout << x;

// Q2
void f(int x){ x=50; }
int main(){ int a=2; f(a); cout<<a; }

// Q3
```

```
// Q4
int a[]={2,4,6};
int* p=a;
cout << *(p+2);

// Q5
string s = "robot";
s.replace(1, 2, "XX");
cout << s;
```

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF

```
void f(int& x){ x=50; }
int main(){ int a=2; f(a); cout<<a; }
```

Q	Answer	Reason
Q1	7	p points to x; *p updates x
Q2	2	x is a copy; a unchanged
Q3	50	x is a reference to a
Q4	6	p+2 → a[2]
Q5	rXXot	Replace 2 chars starting at index 1

Mistakes that lose marks ✘

- Writing a **return type** on a constructor/destructor.
- Using a pointer before initialization, or dereferencing `nullptr`.
- Accessing a **private** member directly from `main`.
- Two overloads that differ only by **return type**.
- Recursion with **no base case** or no progress.
- Thinking pointer arithmetic moves 1 byte, or that `void*` can be dereferenced.
- Out-of-bounds index – last valid index is **size − 1**.
- Using `while(!file.eof())` to read records.
- Dumping a class with **string/pointers** into a binary file.
- Forgetting to deep-copy / destroy a raw owning pointer.

One-page syntax sheet 📌

Task	Syntax / reminder
Class / object	`class X { public: void f(); }; X obj; obj.f();`
Out-of-class definition	`void X::f() { ... }`
Pointer	`int x=5; int* p=&x; cout << *p;`
Pointer to pointer	`int** pp=&p; cout << **pp;`
Object pointer	`X* px=&obj; px->f();`
Reference	`int& r=x; r=8;`

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF

Default argument	`void f(int a, int b = 10);`
Overload	`int f(int); double f(double);`
Static member	`static int count;` → `int X::count = 0;`
Friend function	`friend void inspect(const X&);`
Recursion	`if(base) return value; return step * f(smaller);`
2D array	`int a[rows][cols]; a[r][c];`
String line	`string s; getline(cin, s);`
File input	`ifstream in("a.txt"); while(getline(in,line)){...}`
File output	`ofstream out("a.txt"); out << value << '\n';`
Binary write	`out.write(reinterpret_cast<const char*>(&x), sizeof x);`
Constructor	`X(int v) : member(v) {}`
Copy constructor	`X(const X& other) : member(other.member) {}`
Destructor	`~X() { /* release owned resources */ }`
Member pointer	`int X::* pm=&X::n; obj.*pm; ptr->*pm;`

Glossary 📖

Term	Meaning
Abstraction	Simple interface, hidden details
Address	Location of an object in memory
Array	Fixed-size contiguous sequence of one type
Class	User-defined type with data + functions
Constructor	Initializes an object
Destructor	Runs when object lifetime ends
Encapsulation	Bundling data + operations with access control
File stream	Stream connected to a file
Friend	Non-member given access to private/protected members
Member initializer list	Initializes data members/base classes before body
Object	Instance of a class
Pointer	Variable storing an address
Polymorphism	One name/interface, many behaviours

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF

Reference	Alias of an existing object
Recursion	Function solving a problem by calling itself on a smaller case
Scope	Region where a name is visible
Static member	Belongs to the class, not each object
Manipulator	Formatting/control utility applied to a stream
Undefined behaviour	Standard imposes no requirements on the result

★ Final Study Strategy

First master: **class/object syntax, parameter passing, pointers, strings, file modes, constructor vs destructor**. Then write **at least one full program from each unit** and say the short answers out loud. Your class slides & lecturer's terms are the final exam authority.

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF